



# Assignment: Multi-Source Financial Portfolio Aggregator

## Objective

Build a full-stack application that reads financial data from multiple sources, identifies and removes overlapping records (deduplication), and presents a comprehensive investment portfolio. Your goal is to extract as much value and insight from the provided datasets as possible and display them in a functional, integrated dashboard.

## Data Sources

The application must process the following three datasets:

1. **Order Data:** [Download order.json](#)
2. **MF Central Data:** [Download mf\\_central.json](#)
3. **Account Aggregator (Finarkein):** [Download account\\_aggregator.json](#)

---

## Core Technical Tasks

### 1. Data Processing & Logic

You are responsible for designing the backend logic to handle these files. **No external database is required**; you may read and process the JSON files directly from the local file system.

- **Deduplication (Critical):** If a transaction or holding exists in both MF Central and the Account Aggregator, your logic must ensure it is only counted once. Define a clear tie-break rule (e.g., Account Aggregator data takes precedence).
- **Action Mapping (Buy/Sell):**
  - **Deposits (AA):** Map **DEBIT** to **BUY** (Inflow/Investment) and **CREDIT** to **SELL** (Outflow/Redemption).
  - **Equities (AA):** Use the provided **BUY/SELL** tags.
  - **Mutual Funds (MF Central):** Map **txnSign** where **+** is **BUY** and **-** is **SELL**.

- **Net Worth Calculation:** Calculate the current value across all FI types (Deposits, Equities, Mutual Funds) while ensuring no double-counting of overlapping ISINs or Folios.

## 2. Portfolio Dashboard

Create a functional frontend (React, Angular, or similar) that surfaces the processed data. Rather than following a strict API spec, **show as many data points and insights as you can** to provide a 360-degree view of the user's finances.

**Suggested features to implement:**

- **Total Portfolio Summary:** Aggregated Net Worth and breakdown by asset class.
- **Unified Transaction History:** A single, deduplicated list of activities across all sources.
- **Asset Allocation:** Insights into where the most capital is deployed.
- **Data Integrity:** Clear evidence that duplicate data from different sources has been merged correctly.

---

## Technical Requirements

- **Stack:** Use any modern tech stack (e.g., Django + React, Node + Vue, FastAPI + Plain JS).
- **Containerization:** You must provide a `Dockerfile` and `docker-compose.yml` so the entire project (Frontend + Backend) can be started with a single command.
- **Functional Integrity:** Every feature shown on the UI must be backed by working logic. If a "Total Net Worth" is displayed, it must accurately reflect the deduplicated sum of the JSON files.

---

## Submission Deliverables

1. **Public GitHub Repository:** Containing all source code.
2. **README.md:** \* Clear instructions on how to run the project using **Docker**.
  - Explanation of your **Deduplication Strategy** (how you identified and handled overlapping data).
  - A list of all features/insights you have implemented.
3. **Video Demo Link:** A video (Loom, Drive, or YouTube) showing:
  - The application running.
  - A walkthrough of the code and logic.
  - Specific proof of deduplication (e.g., showing how a Mutual Fund found in two different JSON files is merged into one entry on the UI).

---

## Evaluation Criteria

- **Functional Completeness:** Does the application successfully aggregate and display data from all three sources? Are all implemented features working as intended?
- **Deduplication Accuracy:** Is the logic for removing overlapping Mutual Fund and Transaction data robust?
- **Code Quality & Engineering:** Is the backend logic modular? Is the code clean, readable, and well-organized?
- **DevOps/Setup:** Does the Docker configuration work seamlessly for a "one-click" setup?